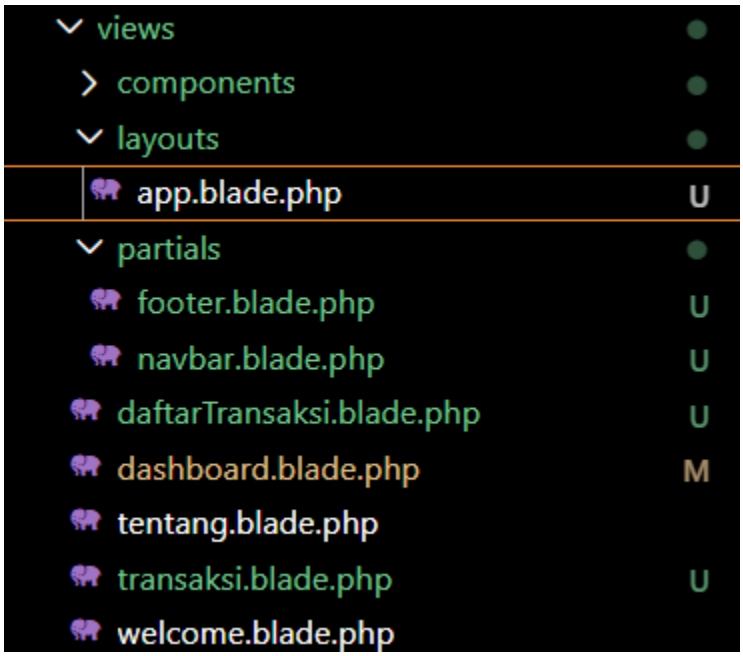
Nama: Neva Maritza Andini
NIM: 242410101021

MANDIRI 3

1. Buat layout master: resources/views/layouts/app.blade.php dengan navbar, main content area (@yield('content')), dan footer

   Jawaban:

   Soal disuruh untuk untuk membuat folder Layout yang berisi navbar, main content area, dan footer di dalam view. Dan diatas saya telat membuat Navbar include('partials.navbar') Area konten utama yield('content') dan Footer include 'partials.footer'.

2. Buat partial: resources/views/partials/navbar.blade.php dan sisipkan dengan @include di layout
   Ini halaman Layouts yang isinya struktur html:

resources > views > layouts > app.blade.php > ...

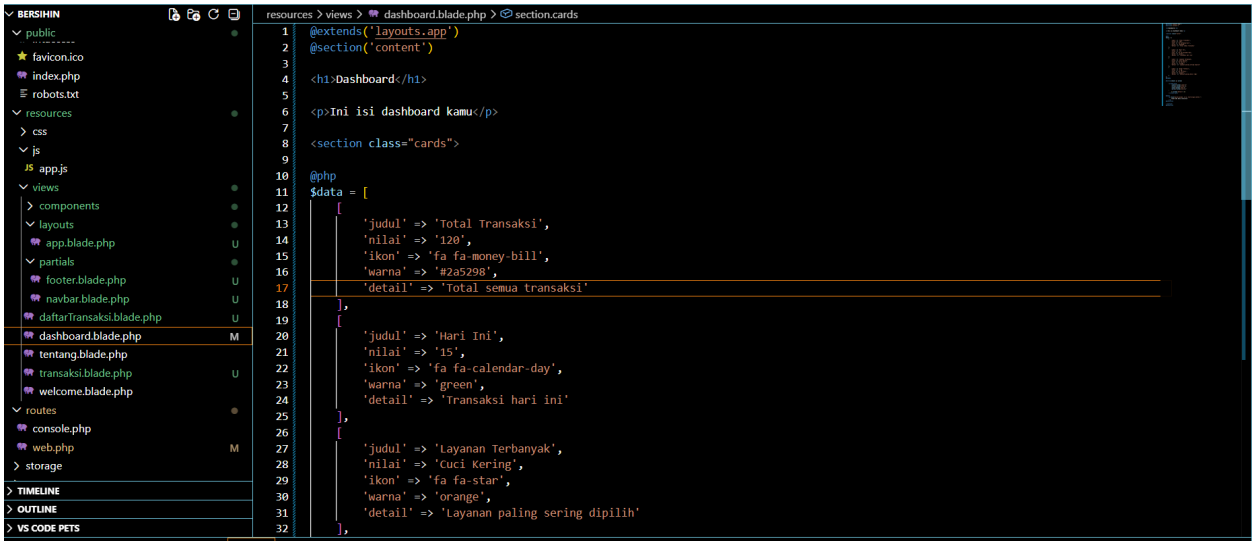
```php
<!DOCTYPE html>
<html>
<head>
    <title>App</title>

    <link rel="stylesheet" href="{{ asset('css/style.css') }}">
    <script src="https://unpkg.com/@phosphor-icons/web"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.2/css/all.min.css">
    <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-awesome/4.2.0/css/font-awesome.min.css">
    <link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Cookie" type="text/css">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>

    @include('partials.navbar')

    @if(session('success'))
    <div>
        {{ session('success') }}
    </div>
    @endif

    <main>
        @yield('content')
    </main>

    @include('partials.footer')

    <script src="{{ asset('js/script.js') }}"></script>
@stack('scripts')
</body>
</html>
```

Nah di app ini sudah di include kan navbar, aku membuat navbar bukan layout. Maaf kalau kebalik bu shinta tapi ini yang ada di web saya seperti ini.
Ini Navbar:

resources > views > partials > navbar.blade.php > header.navbar > nav > a

```header.navbar
<header class="navbar">
  <div class="logo">
    <img src="image/logoBersih.png" alt="logo">
  </div>
  <div class="box">
        <form class="search-box">
          <input type="text" placeholder="Cari disini">
          <button type="submit" class="icons">
            <i class="fas fa-search"></i>
          </button>
        </form>
      </div>
  <nav>
    <a href="#">Dashboard</a>
    <a href="transaksi.html">Transaksi</a>
    <a href="#">Daftar Transaksi</a>
    <a href="#">Pelanggan</a>
  </nav>
</header>
```

3. Buat halaman dashboard: resources/views/dashboard.blade.php yang @extends('layouts.app')



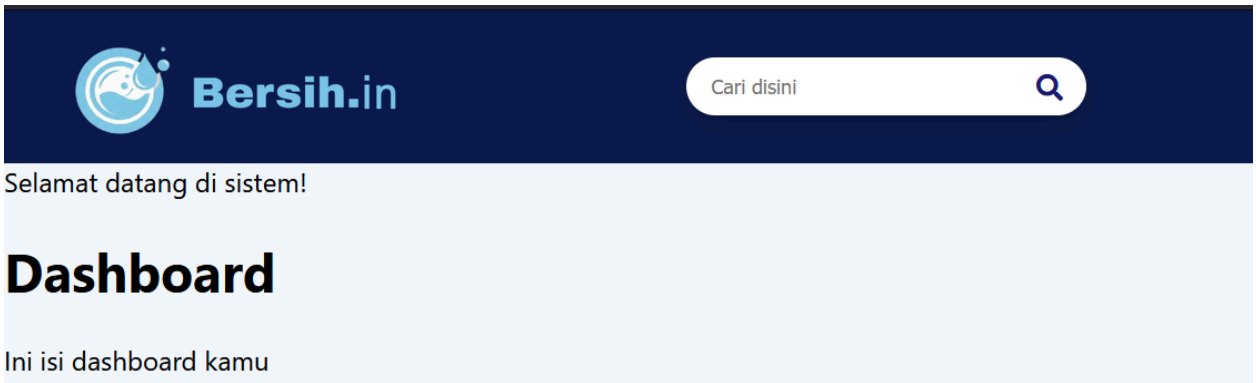
Sudah dibuat dashboard.blade.php yang di dalamnya ada extends ke layouts jadi dashboard ini akan memakai struktur yang ada di layout.

4. Tampilkan flash session message di layout: @if(session('success')) ... @endif

Di dalam Layouts.app sudah ada flash session message, yang akan terhubung ke routes dan nanti akan ditampilkan di web

```php
Route::get('/test-flash', function () {
    return redirect('/dashboard')
        ->with('success', 'Selamat datang di sistem!');
});
Route::view('/transaksi', 'transaksi');
Route::view('/daftar-transaksi', 'daftar-transaksi');
```

Seperti ini tampilannya:



5. Buat komponen: resources/views/components/stat-card.blade.php (kartu statistik dengan props: judul, nilai, ikon, warna)

```blade
<div class="card" onclick="flipCard(this)">
    <div class="card-inner">
        <div class="card-front" style="border-top: 5px solid {{ $warna }}">
            <i class="{{ $ikon }}"></i>
            <h3>{{ $judul }}</h3>
            <p>{{ $nilai }}</p>
        </div>

        <div class="card-back">
            <h4>Detail</h4>
            <p>{{ $slot }}</p>
        </div>

    </div>
</div>
```

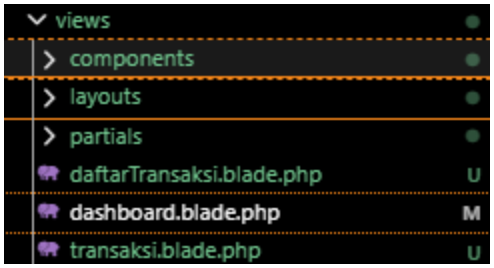
Diview sudah dibuat components, yang isinya stat-card yang nantinya dapat di panggil di dashboard.

6. Gunakan @forelse di dashboard untuk loop array data dummy (jumlah mahasiswa, mata kuliah, dll)

```
10  $data = [
11      [
12          'judul' => 'Total Transaksi',
13          'nilai' => '120',
14          'ikon' => 'fa fa-money-bill',
15          'warna' => '#2a5298',
16          'detail' => 'Total semua transaksi'
17      ],
18      [
19          'judul' => 'Hari Ini',
20          'nilai' => '15',
21          'ikon' => 'fa fa-calendar-day',
22          'warna' => 'green',
23          'detail' => 'Transaksi hari ini'
24      ],
25      [
26          'judul' => 'Layanan Terbanyak',
27          'nilai' => 'Cuci Kering',
28          'ikon' => 'fa fa-star',
29          'warna' => 'orange',
30          'detail' => 'Layanan paling sering dipilih'
31      ],
32      [
33          'judul' => 'Order Terkecil',
34          'nilai' => '2 Kg',
35          'ikon' => 'fa fa-users',
36          'warna' => 'purple',
37          'detail' => 'Orderan paling kecil (Kg)'
38      ]
39  ];
40  @endphp
41
42  @forelse($data as $item)
43
44      <x-stat-card
45          :judul="$item['judul']"
46          :nilai="$item['nilai']"
47          :ikon="$item['ikon']"
48          :warna="$item['warna']"
49      >
50          {{ $item['detail'] }}
51      </x-stat-card>
52
53  @empty
54      <p style="grid-column: 1/-1; text-align:center;">
55          Tidak ada data statistik
56      </p>
57  @endforelse
```

Penggunaan @forelse di Blade Laravel untuk menampilkan data dashboard dalam bentuk card berdasarkan array data dummy. Data tersebut berisi beberapa informasi seperti jumlah transaksi, transaksi hari ini, layanan terbanyak, dan order terkecil yang disimpan dalam array $data. Kemudian, @forelse digunakan untuk melakukan perulangan setiap item dan mengirimkan nilainya ke komponen <x-stat-card> agar ditampilkan secara dinamis. Jika data tersedia, maka semua card akan muncul sesuai isi array, sedangkan jika data kosong, bagian @empty akan dijalankan dan menampilkan pesan “Tidak ada data statistik”. Dengan cara ini, penggunaan @forelse tidak hanya berfungsi untuk looping, tetapi juga memastikan tampilan tetap aman ketika tidak ada data yang ditampilkan.

7. Buat halaman 'Tentang' dan 'Kontak' yang @extends layout yang sama



Saya bukan membuat halaman tentang dan kontak tapi saya membuat halaman daftarTransaksi, dan Transaksi sesuai dengan kebutuhan web saya.

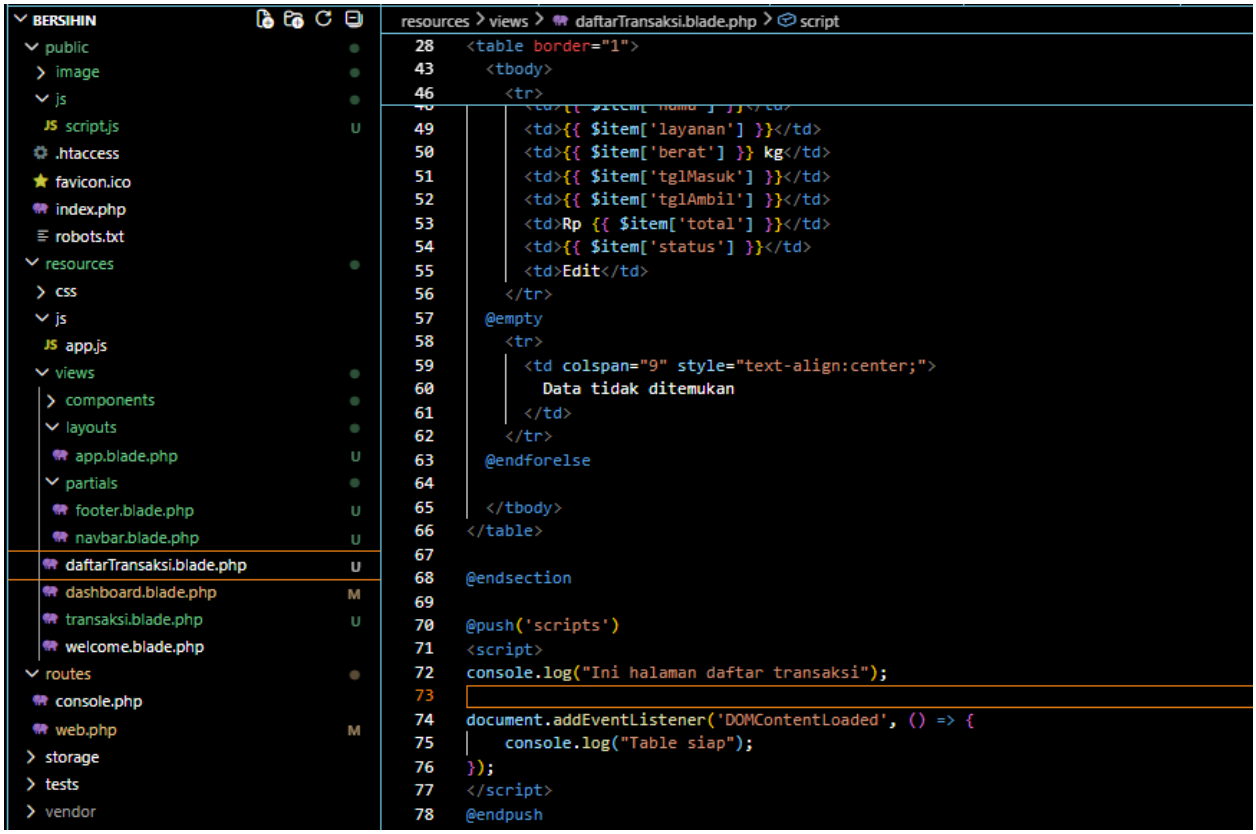
```
@extends('layouts.app')
@section('title', 'Transaksi')
@section('content')

<div class="transaksi-page">
  <div class="container">

    <div class="price-list">
      <h3>Daftar Layanan</h3>

      <div class="price-card">
        <p>Cuci</p>
        <span>Rp 5.000 / kg</span>
      </div>

      <div class="price-card">
        <p>Cuci + Setrika</p>
        <span>Rp 7.000 / kg</span>
      </div>

      <div class="price-card">
        <p>Setrika</p>
        <span>Rp 4.000 / kg</span>
      </div>
    </div>

    <div class="form-transaksi">
      <h3>Form Transaksi</h3>
```

Ini yang diatas merupakan halaman transaksi dan sudah extend ke layouts seperti yang di minta, lalu di bawah ini merupakan halaman daftar transaksi yang juga sudah extend ke layouts.

```php
@extends('layouts.app')
@section('title', 'Daftar Transaksi')
@section('content')
<h2>Daftar Transaksi</h2>
@php
$transaksi = [
    [
        'nama' => 'Andi',
        'layanan' => 'Cuci',
        'berat' => 2,
        'tglMasuk' => '2026-05-01',
        'tglAmbil' => '2026-05-03',
        'total' => 10000,
        'status' => 'Diproses'
    ],
    [
        'nama' => 'Budi',
        'layanan' => 'Setrika',
        'berat' => 3,
        'tglMasuk' => '2026-05-02',
        'tglAmbil' => '2026-05-04',
        'total' => 12000,
        'status' => 'Selesai'
    ]
];
@endphp
```

8. Gunakan @push('scripts') dan @stack('scripts') untuk menambahkan JavaScript spesifik per halaman

```php
<div class="transaksi-page">
  <div class="container">
    <div class="form-transaksi">
    </div>

  </div>
</div>

@endsection

@push('scripts')
<script>
const layanan = document.querySelector('[name="layanan"]');
const berat = document.querySelector('[name="berat"]');
const total = document.querySelector('[placeholder="Total Harga (Auto)"]');

function hitungTotal() {
    let harga = layanan.value;
    let kg = berat.value;

    if (harga && kg) {
        total.value = "Rp " + (harga * kg);
    }
}

layanan.addEventListener('change', hitungTotal);
berat.addEventListener('input', hitungTotal);
</script>
@endpush
```



Pada halaman transaksi dan daftar transaksi telah saya berikan @push(‘scripts’) untuk mengirimkan scripts yang akan di tangkap oleh dashboard

```
web.php M | stat-card.blade.php U | app.blade.php U | navbar.blade.php U | footer.blade.php U | style.css U | script
resources > views > layouts > app.blade.php > ...
2   <html>
3   <head>
6       <link rel="stylesheet" href="{{ asset('css/style.css') }}">
7       <script src="https://unpkg.com/@phosphor-icons/web"></script>
8       <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.2/css/all.min.css">
9       <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-awesome/4.2.0/css/font-awesome.min.css">
10      <link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Cookie" type="text/css">
11      <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
12  </head>
13  <body>
14
15      @include('partials.navbar')
16
17      @if(session('success'))
18      <div>
19          {{ session('success') }}
20      </div>
21      @endif
22
23      <main>
24          @yield('content')
25      </main>
26
27      @include('partials.footer')
28
29      <script src="{{ asset('js/script.js') }}"></script>
30  @stack('scripts')
31  </body>
32  </html>
```

@stackk(‘Script’) untuk menangkap script yang telah dikirim di transaksi dan daftar Transaksi.

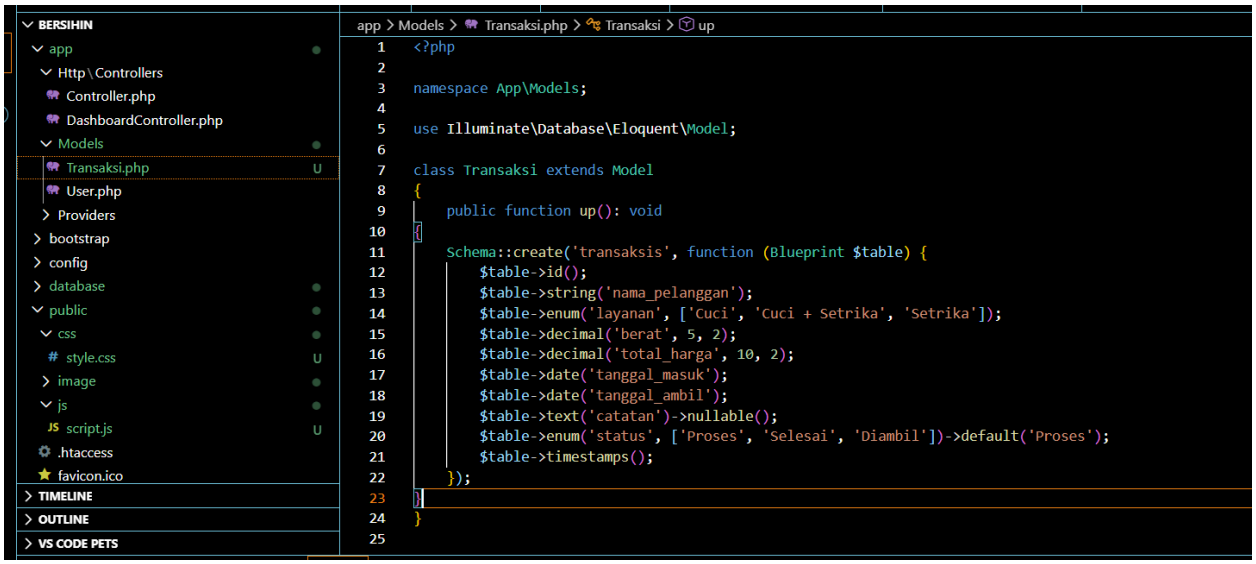
Mandiri 4

1. Buat Model dan Migration sekaligus: php artisan make:model Mahasiswa -m

```
BERSIHIN
app
Http\Controllers
Controller.php
DashboardController.php
Models
Transaksi.php U
User.php
TIMELINE
OUTLINE
VS CODE PETS

app > Models > Transaksi.php > ...
1   <?php
2
3   namespace App\Models;
4
5   use Illuminate\Database\Eloquent\Model;
6
7   class Transaksi extends Model
8   {
9       //
10  }
11

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   DEVDB

PS C:\laragon\www\BersihIn>
PS C:\laragon\www\BersihIn> php artisan make:model Transaksi -m

  INFO  Model [C:\laragon\www\BersihIn\app\Models\Transaksi.php] created successfully.

  INFO  Migration [C:\laragon\www\BersihIn\database\migrations\2026_05_05_150813_create_transaksis_table.php] created successfully.

PS C:\laragon\www\BersihIn>
```

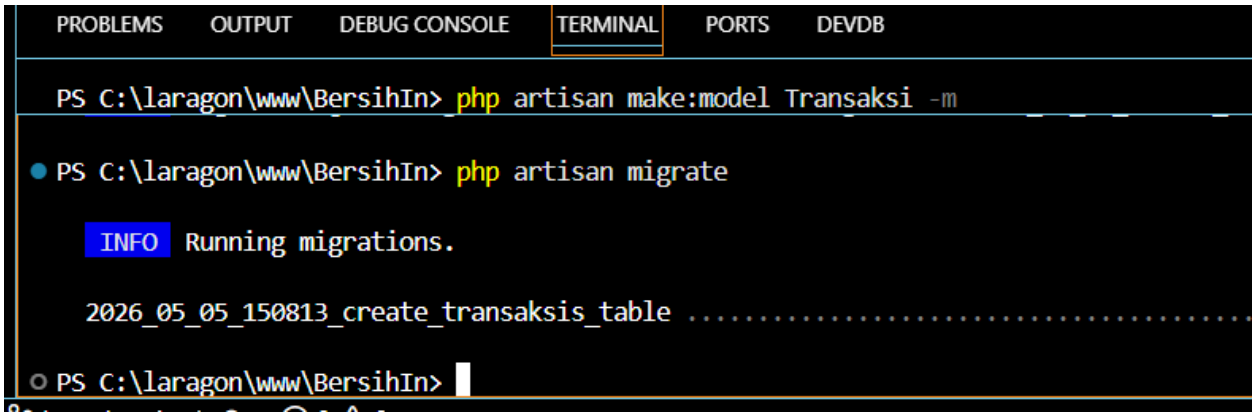
Sudah dibuat sebuah model namu tidak sama seperti bu shinta suruh karena ini menyesuaikan web saya bu. Saya membuat model untuk transaksi

2. Edit file migration: tambahkan kolom nim, nama, email, jurusan (enum), ipk (decimal), semester, aktif (boolean), foto (nullable)

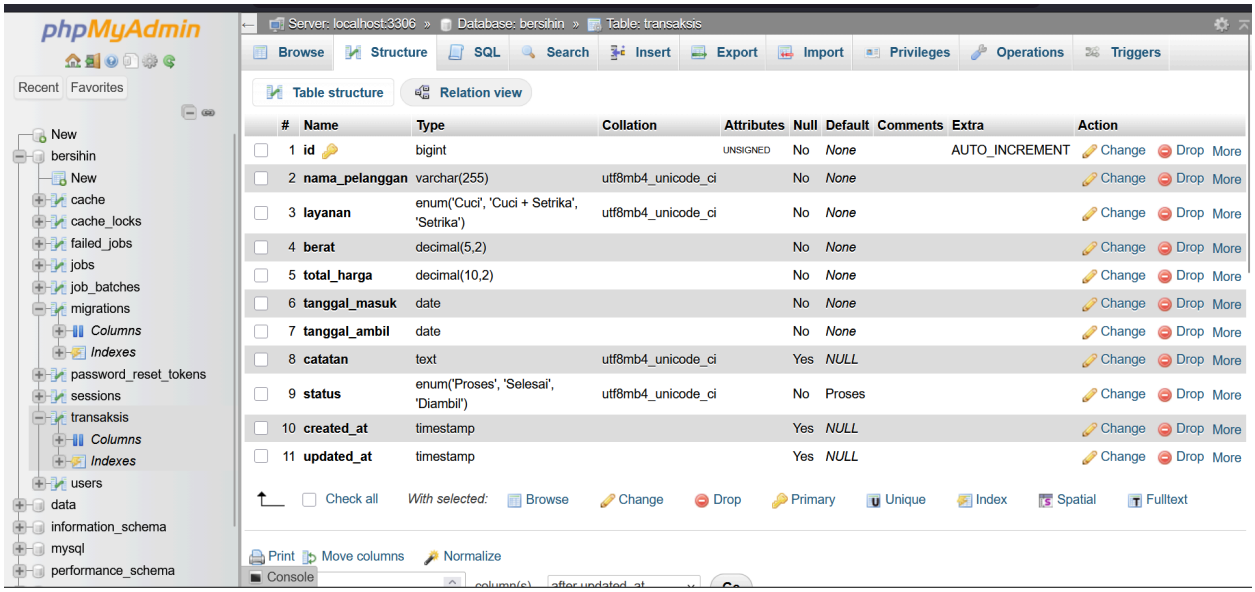


Pada file migration, kolom-kolom ditambahkan sesuai dengan kebutuhan sistem transaksi laundry pada aplikasi Bersihin. Kolom yang dibuat meliputi nama_pelanggan, layanan, berat, total_harga, tanggal_masuk, tanggal_ambil, catatan, dan status. Setiap kolom menggunakan tipe data yang disuruh pada soal, seperti string untuk nama, enum untuk layanan dan status, decimal untuk berat dan total harga, serta date untuk tanggal. Dengan struktur ini, data transaksi laundry dapat disimpan dan dikelola dengan baik sesuai fitur form transaksi dan daftar transaksi pada aplikasi.

3. Jalankan migration: php artisan migrate — cek di phpMyAdmin bahwa tabel sudah terbuat



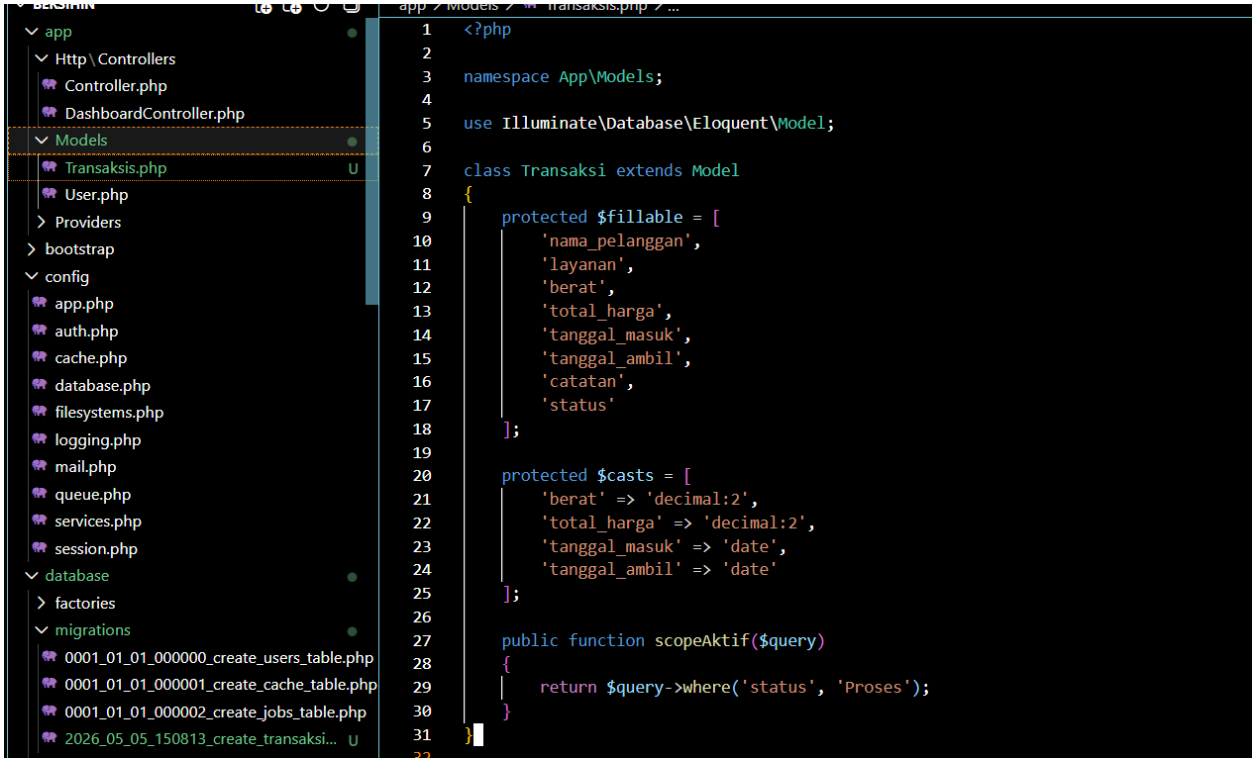
Perintah php artisan migrate digunakan untuk mengeksekusi file migration sehingga tabel transaksis berhasil dibuat di database. Setelah itu dilakukan dicek melalui phpMyAdmin untuk memastikan bahwa tabel beserta seluruh kolomnya telah terbentuk.



Dan ini hasilnya setelah di cek, dan ini pada file migrationnya.

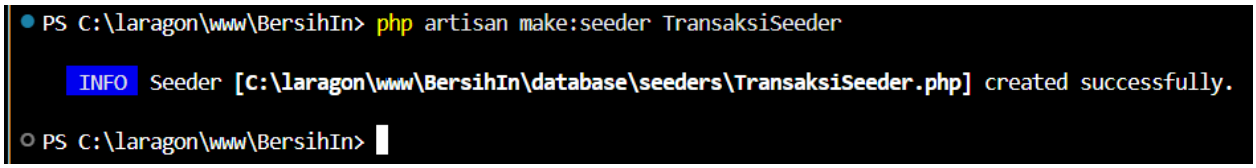
```php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('transaksis', function (Blueprint $table) {
            $table->id();
            $table->string('nama_pelanggan');
            $table->enum('layanan', ['Cuci', 'Cuci + Setrika', 'Setrika']);
            $table->decimal('berat', 5, 2);
            $table->decimal('total_harga', 10, 2);
            $table->date('tanggal_masuk');
            $table->date('tanggal_ambil');
            $table->text('catatan')->nullable();
            $table->enum('status', ['Proses', 'Selesai', 'Diambil'])->default('Proses');
            $table->timestamps();
        });
    }
```

4. Edit Model Mahasiswa: isi $fillable, tambahkan $casts, dan buat local scope scopeAktif()

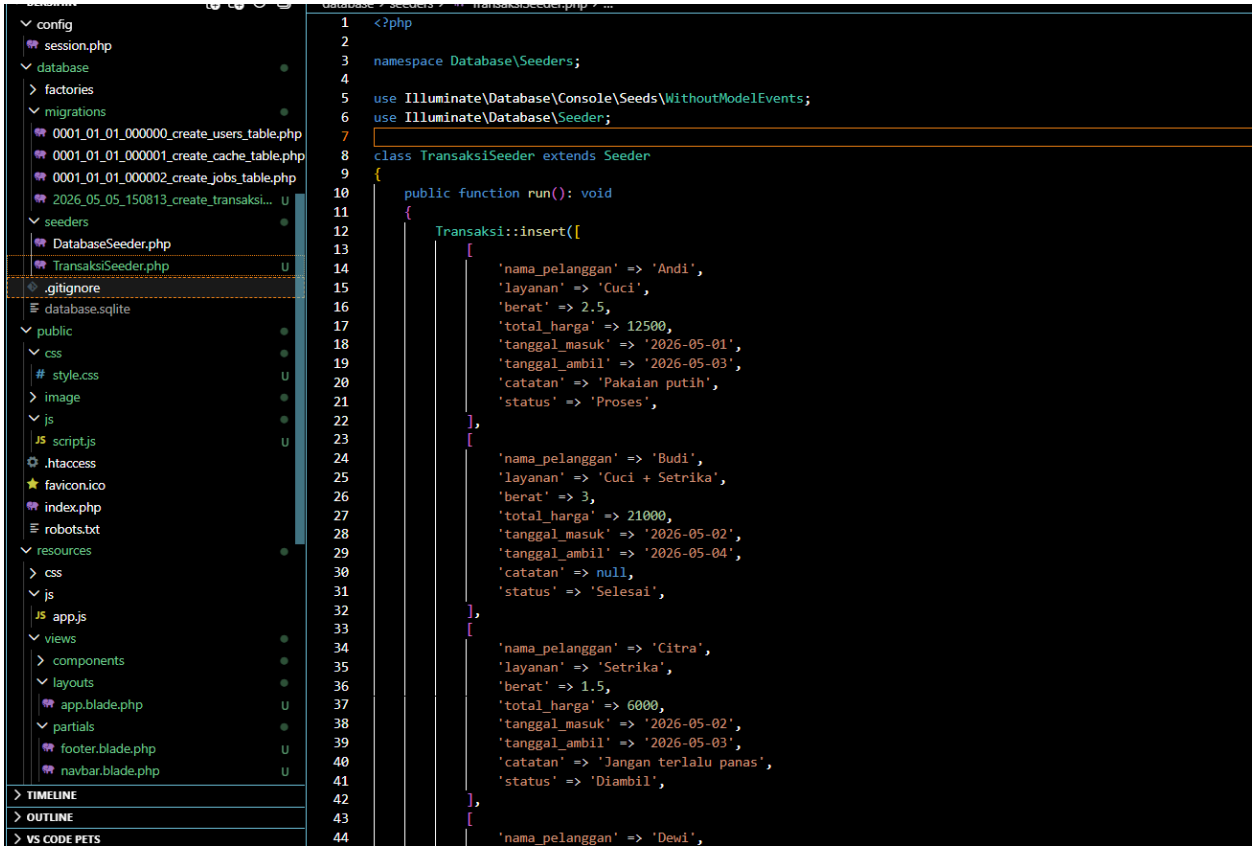


Pada model Transaksi, properti $fillable digunakan untuk menentukan kolom yang dapat diisi secara mass assignment, seperti nama pelanggan, layanan, berat, dan lainnya. Properti $casts digunakan untuk mengatur tipe data agar sesuai dengan kebutuhan, misalnya mengubah berat dan total harga menjadi decimal serta tanggal menjadi format date. Selain itu, dibuat local scope scopeAktif() untuk mempermudah pengambilan data transaksi yang masih dalam status proses.

5. Buat seeder: php artisan make:seeder MahasiswaSeeder — isi dengan 5-10 data dummy

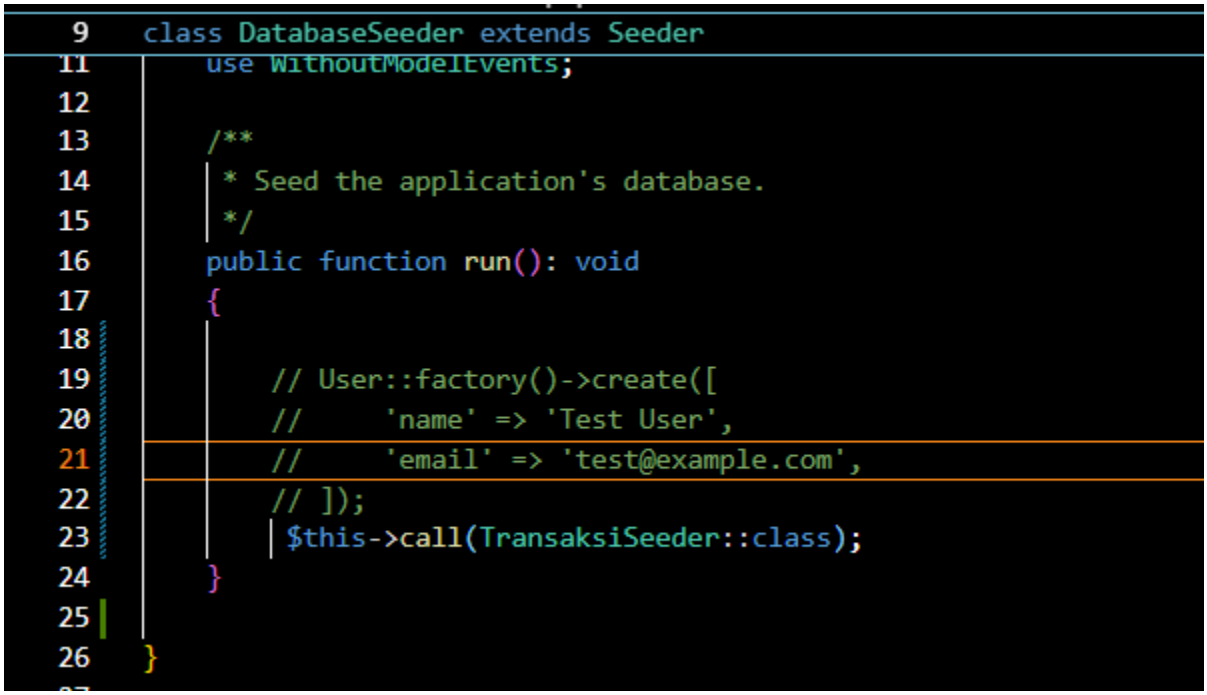


Pertama dibuat dulu TransaksiSeedernya kemudian



Diisi TransaksiSeeder dengan data dummy, disini saya masukin 5 data dummy.

6. Daftarkan seeder di DatabaseSeeder.php dan jalankan: php artisan db:seed

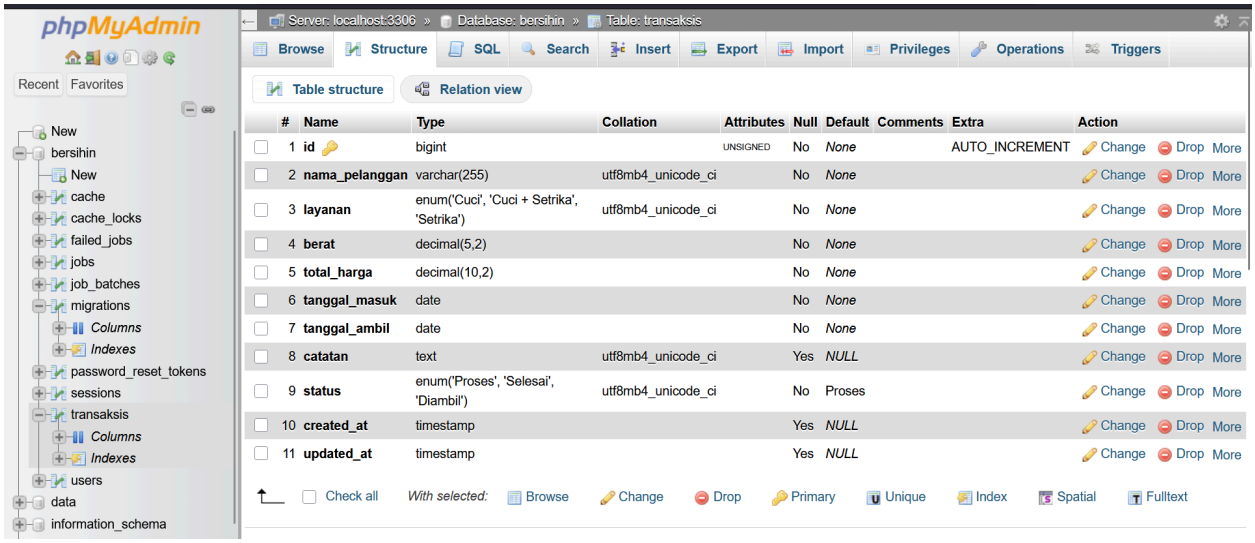


Ini di DataSeeder dipanggil TransaksiSeeder.

Kemudian setelah itu ketik di terminal php artisan db:seed, agar datanya bisa masuk ke phpMyAdmin.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   DEVDB

  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... RUNNING
● PS C:\laragon\www\BersihIn> php artisan db:seed

  INFO  Seeding database.

  Database\Seeders\TransaksiSeeder ..... RUNNING
  Database\Seeders\TransaksiSeeder ..... 257 ms DONE
```

Dan datanya sudah masuk di phpMyAdmin



7. Buka php artisan tinker dan coba query: Mahasiswa::all(), Mahasiswa::where('jurusan','SI')->get(), Mahasiswa::avg('ipk')

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVDB

```
PS C:\laragon\www\BersihIn> php artisan tinker
Psy Shell v0.12.22 (PHP 8.3.30 — cli) by Justin Hileman
New PHP manual is available (latest: 3.0.5). Update with `doc --update-manual`

> use App\Models\Transaksi;

> Transaksi::all();

= Illuminate\Database\Eloquent\Collection {#8079
    all: [
      App\Models\Transaksi {#8074
        id: 1,
        nama_pelanggan: "Andi",
        layanan: "Cuci",
        berat: "2.50",
        total_harga: "12500.00",
        tanggal_masuk: "2026-05-01",
        tanggal_ambil: "2026-05-03",
        catatan: "Pakaian putih",
        status: "Proses",
        created_at: null,
        updated_at: null,
      },
      App\Models\Transaksi {#8073
        id: 2,
        nama_pelanggan: "Budi",
        layanan: "Cuci + Setrika",
        berat: "3.00",
        total_harga: "21000.00",
        tanggal_masuk: "2026-05-02",
        tanggal_ambil: "2026-05-04",
        catatan: null,
        status: "Selesai",
        created_at: null,
        updated_at: null,
      },
      App\Models\Transaksi {#8076
        id: 3,
        nama_pelanggan: "Citra",
        layanan: "Setrika",
        berat: "1.50",
        total_harga: "6000.00",
        tanggal_masuk: "2026-05-02",
        tanggal_ambil: "2026-05-03",
        catatan: "Jangan terlalu panas",
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    DEVDB

PS C:\laragon\www\BersihIn> php artisan tinker
        tanggal_masuk: "2026-05-04",
        tanggal_ambil: "2026-05-06",
        catatan: "Baju kerja",
        status: "Selesai",
        created_at: null,
        updated_at: null,
      },
    ],
  }

> Transaksi::where('layanan', 'Cuci')->get();

= Illuminate\Database\Eloquent\Collection {#7386
    all: [
      App\Models\Transaksi {#8107
        id: 1,
        nama_pelanggan: "Andi",
        layanan: "Cuci",
        berat: "2.50",
        total_harga: "12500.00",
        tanggal_masuk: "2026-05-01",
        tanggal_ambil: "2026-05-03",
        catatan: "Pakaian putih",
        status: "Proses",
        created_at: null,
        updated_at: null,
      },
      App\Models\Transaksi {#7389
        id: 4,
        nama_pelanggan: "Dewi",
        layanan: "Cuci",
        berat: "4.00",
        total_harga: "20000.00",
        tanggal_masuk: "2026-05-03",
        tanggal_ambil: "2026-05-05",
        catatan: null,
        status: "Proses",
        created_at: null,
        updated_at: null,
      },
    ],
  }
```

```
> Transaksi::avg('total_harga');

= "14700.000000"

>
```

Laravel Tinker untuk mengecek dan mengolah data pada model Transaksi. Perintah Transaksi::all() digunakan untuk menampilkan seluruh data yang sudah berhasil di-*seed* ke database. Kemudian, Transaksi::where('layanan', 'Cuci')->get() digunakan untuk memfilter data berdasarkan layanan “Cuci”. Terakhir, Transaksi::avg('total_harga') menghitung rata-rata total harga transaksi, dengan hasil 14700.

8. Coba buat relasi: buat tabel mata_kuliahs dan tabel pivot krs, lalu definisikan relasi belongsToMany di Model

```
EXPLORER                                   pt.js U   dashboard.blade.php M   transaksi.blade.php U   daftarTransaksi.blade.php U   Trans
BERSIHIN                                   database > seeders > DatabaseSeeder.php > DatabaseSeeder > run
app                                        9   class DatabaseSeeder extends Seeder
Http\Controllers                           11      use WithoutModelEvents;
Controller.php                             12
DashboardController.php                    13      /**
Models                                     14       * Seed the application's database.
Layanan.php                U               15       */
Transaksi.php              U               16      public function run(): void
User.php                                   17      {
Providers                                  18
TIMELINE                                   19          // User::factory()->create([
OUTLINE                                    20          //     'name' => 'Test User',
VS CODE PETS                               21          //     'email' => 'test@example.com',
                                           22          // ]);
                                           23          $this->call(TransaksiSeeder::class);
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   DEVDB
PS C:\laragon\www\BersihIn> php artisan make:model Layanan -m

  INFO  Model [C:\laragon\www\BersihIn\app\Models\Layanan.php] created successfully.

  INFO  Migration [C:\laragon\www\BersihIn\database\migrations\2026_05_05_164131_create_layanans_table.php] created successfully.

PS C:\laragon\www\BersihIn>
```

Perintah ini digunakan untuk membuat file yang akan digunakan untuk membuat model layanan.

```
BERSIHIN                                   database > migrations > 2026_05_05_164131_create_layanans_table.php
database                                   6
migrations                                 7   return new class extends Migration
0001_01_01_000002_create_jobs_table.php    8   {
2026_05_05_150813_create_transaksi...  U   9
2026_05_05_164131_create_layanans...  U   10      public function up(): void
2026_05_05_164307_create_transaksi...  U   11      {
seeders                                    12          Schema::create('layanans', function (Blueprint $table) {
DatabaseSeeder.php                     M   13          $table->id();
TIMELINE                                   14          $table->string('nama_layanan');
OUTLINE                                    15          $table->integer('harga_per_kg');
VS CODE PETS                               16          $table->timestamps();
                                           17      });
                                           18      }
```

Tabel layanans yang digunakan untuk menyimpan data layanan laundry, seperti nama layanan dan harga per kilogram.

```
PS C:\laragon\www\BersihIn> php artisan make:migration create_transaksi_layanan_table

  INFO  Migration [C:\laragon\www\BersihIn\database\migrations\2026_05_05_164307_create_transaksi_layanan_table.php] created successfully.

PS C:\laragon\www\BersihIn>
```

Perintah ini digunakan untuk membuat file migration yang akan digunakan untuk membuat tabel pivot transaksi_layanan.

```
BERSIHIN                                   database > migrations > 2026_05_05_164307_create_transaksi_layanan_table.php
database                                   7   return new class extends Migration
migrations                                 8   {
0001_01_01_000002_create_jobs_table.php    9       public function up(): void
2026_05_05_150813_create_transaksi...  U   10      {
2026_05_05_164131_create_layanans...  U   11          Schema::create('transaksi_layanan', function (Blueprint $table) {
2026_05_05_164307_create_transaksi...  U   12          $table->id();
seeders                                    13          $table->foreignId('transaksi_id')->constrained()->cascadeOnDelete();
DatabaseSeeder.php                     M   14          $table->foreignId('layanan_id')->constrained()->cascadeOnDelete();
TIMELINE                                   15          $table->timestamps();
                                           16      });
                                           17      }
```

Isi migration untuk tabel transaksi_layanan, yang berfungsi sebagai tabel pivot. Tabel ini berisi kolom transaksi_id dan layanan_id sebagai foreign key yang menghubungkan tabel transaksis dan layanans, sehingga membentuk relasi many-to-many.

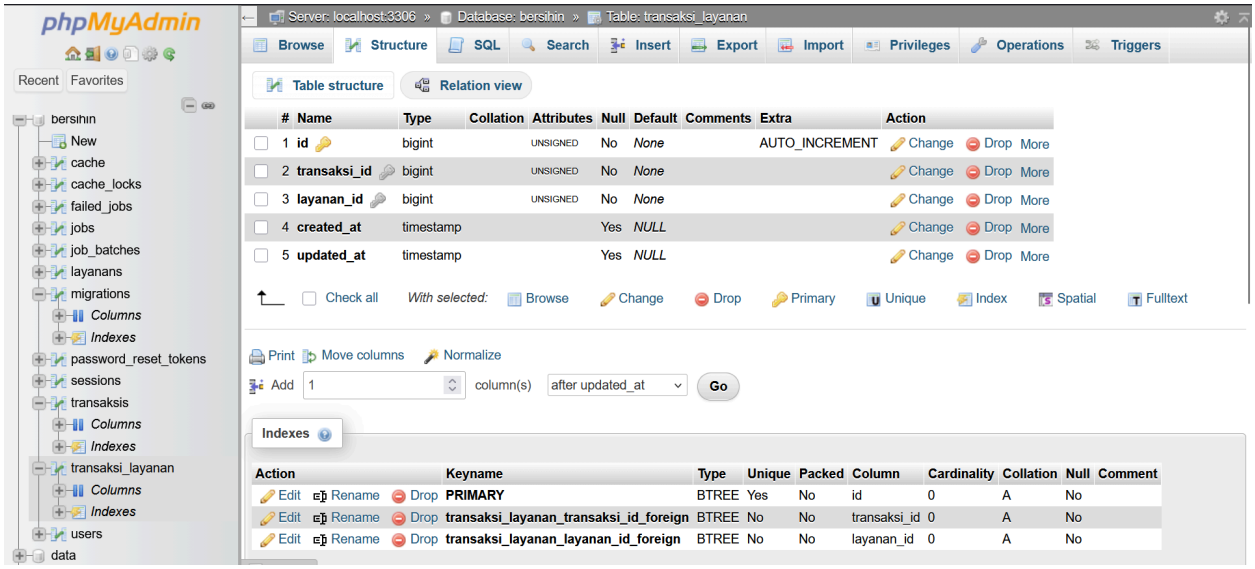
```
DashboardController.php
Models
Layanan.php U
Transaksi.php U
User.php
Providers
bootstrap
config
app.php
```

```
29        return $query->where('status', 'Proses');
30    }
31    public function layanans()
32    {
33        return $this->belongsToMany(Layanan::class, 'transaksi_layanan');
34    }
35 }
36
```

```
DashboardController.php
Models
Layanan.php U
Transaksi.php U
User.php
Providers
bootstrap
config
app.php
```

```
5  use Illuminate\Database\Eloquent\Model;
6
7  class Layanan extends Model
8  {
9      public function transaksis()
10     {
11         return $this->belongsToMany(Transaksi::class, 'transaksi_layanan');
12     }
13 }
14
```

Ini untuk tampilannya:



Relasi many-to-many antara Transaksi dan Layanan berhasil dibuat menggunakan tabel pivot transaksi_layanan. Dapat dilihat pada gambar diatas dengan adanya foreign key transaksi_id dan layanan_id yang menghubungkan tabel transaksis dan layanans. Relasi juga didefinisikan pada model menggunakan method belongsToMany, sehingga data antar tabel dapat saling terhubung dengan baik.